# Proposal Pengembangan dan Uji Coba GraphMethod AI Pull Request Reviewer

**Ditujukan kepada:** Tim Developer dan Engineering Lead SMP Technology
**Pengusul:** [Nama pengusul], Peserta Magang
**Tanggal:** 13 Oktober 2026
**Status:** Usulan pengembangan dan pilot internal

## 1. Ringkasan proposal

Saya mengusulkan pengembangan dan uji coba **GraphMethod AI Pull Request Reviewer** sebagai alat bantu peninjauan kode untuk tim developer SMP Technology. Sistem terintegrasi dengan GitHub dan menghasilkan ringkasan perubahan, analisis dampak, komentar pada baris kode, serta saran perbaikan langsung pada Pull Request (PR).

Pembeda utama sistem adalah penggunaan **structural graph retrieval**: hubungan antarfungsi, kelas, dan pengujian dipetakan ke Neo4j, lalu potongan kode yang berhubungan dengan perubahan dipilih sebagai konteks bagi model AI. Dengan demikian, model memperoleh informasi tambahan mengenai kode di luar diff PR.

Codebase saat ini telah memiliki implementasi dasar berupa GitHub App, webhook, antrean pekerjaan, worker, indexer Python, penyimpanan graph, integrasi Gemini, dan publikasi review. Proposal ini melanjutkan implementasi tersebut melalui enam pengembangan utama: dukungan multi-bahasa, perintah `/review`, konfigurasi per repository, laporan cakupan review, pelacakan penyelesaian temuan, serta analisis graph sebelum dan sesudah perubahan.

Penerapan diusulkan melalui pilot terbatas pada repository yang ditentukan perusahaan. Hasil review AI menjadi masukan bagi developer dan reviewer manusia. Keputusan penerapan saran, approval, dan merge tetap berada pada tim. Manfaat terhadap waktu review dan kualitas temuan akan diukur selama pilot, bukan diasumsikan sudah terbukti.

# 2. Latar belakang dan tujuan

Perubahan kode dapat berdampak pada bagian lain yang tidak terlihat dalam diff, misalnya pemanggil fungsi, kelas turunan, atau pengujian. Memahami dampak tersebut membutuhkan konteks repository dan waktu reviewer. AI yang hanya membaca diff juga berpotensi kehilangan konteks yang sama.

Sistem ini ditujukan untuk membantu pemeriksaan awal, memperjelas dampak perubahan, dan menyediakan masukan yang dapat diperiksa reviewer. Relevansi masalah tersebut terhadap alur kerja SMP Technology akan dikonfirmasi melalui pemilihan repository pilot dan diskusi dengan pembimbing atau engineering lead.

Tujuan pengembangan:

1. Memberikan umpan balik awal otomatis pada PR melalui GitHub.
2. Menambahkan konteks hubungan kode untuk membantu identifikasi potensi bug dan ketidakcocokan perilaku.
3. Mendukung bahasa yang digunakan pada repository pilot secara bertahap.
4. Memungkinkan pengaturan review sesuai kebutuhan setiap repository.
5. Menunjukkan cakupan analisis dan perkembangan temuan secara transparan.
6. Mengukur ketepatan temuan, keandalan proses, waktu analisis, dan penggunaan token.

# 3. Gambaran sistem dan alur penggunaan

GitHub menjadi antarmuka utama; developer tidak perlu berpindah ke dashboard terpisah.

| Tahap | Proses |
|---|---|
| 1 | Developer membuka atau memperbarui PR. Setelah fitur tambahan selesai, developer berwenang juga dapat meminta review melalui `/review`. |
| 2 | GitHub mengirim webhook ke FastAPI. Sistem memvalidasi tanda tangan webhook dan mencatat pekerjaan dalam PostgreSQL. |
| 3 | Worker mengambil metadata PR, diff, dan sumber kode pada commit yang dianalisis. |

| Tahap | Proses |
|---|---|
| 4 | Indexer membentuk graph kode. Saat ini hanya Python; pengembangan akan menambahkan bahasa yang disepakati untuk pilot. |
| 5 | Sistem memilih simbol yang berubah dan kode terkait dari Neo4j. Pengembangan akan menambahkan konteks versi sebelum perubahan. |
| 6 | Prompt berisi instruksi review, metadata PR, diff, dan konteks terpilih dikirim ke Gemini. |
| 7 | Jawaban terstruktur diperiksa, lalu diterbitkan sebagai ringkasan dan komentar review GitHub. |
| 8 | Developer memeriksa temuan dan memperbarui kode. Fitur pelacakan akan menghubungkan temuan dengan hasil analisis berikutnya. |

Contoh ilustratif: sebuah PR mengubah parameter fungsi pemrosesan pesanan. Graph dapat membantu mengambil kode pemanggil fungsi tersebut agar model menilai apakah pemanggilan masih sesuai. Temuan harus menyebutkan lokasi dan alasan yang dapat diperiksa developer; contoh ini bukan hasil pengujian pada repository perusahaan.

Komponen utama adalah Python/FastAPI untuk API dan worker, PostgreSQL untuk pekerjaan serta catatan review, Neo4j untuk graph, Gemini untuk analisis, dan GitHub API untuk integrasi. Sistem menggunakan prompting dan retrieval; pengembangan ini tidak memerlukan pelatihan model AI sendiri.

# 4. Inventaris fitur yang sudah tersedia

**Arti status:** “Tersedia” berarti ditemukan implementasinya dalam codebase yang diperiksa pada 13 Oktober 2026. Status ini tidak menyatakan bahwa fitur telah lolos uji integrasi di lingkungan SMP Technology atau siap produksi.

| No. | Fitur tersedia | Kemampuan dan batas saat ini |
|---|---|---|
| 1 | Integrasi GitHub App | Autentikasi aplikasi dan installation token untuk mengakses repository sesuai izin pemasangan. |

| No. | Fitur tersedia | Kemampuan dan batas saat ini |
|---|---|---|
| 2 | Review otomatis berdasarkan event PR | Memproses `opened`, `synchronize`, `reopened`, dan `ready_for_review`. Push tanpa PR bukan pemicu review. |
| 3 | Validasi webhook | Memeriksa tanda tangan HMAC SHA-256 sebelum memproses event. |
| 4 | Antrean pekerjaan PostgreSQL | Mencatat job, mengklaim pekerjaan dengan penguncian, serta mencegah duplikasi delivery/job melalui constraint. |
| 5 | Worker dan pengulangan pekerjaan gagal | Menjalankan analisis di luar request webhook, mencatat kegagalan, dan mencoba ulang sampai batas konfigurasi. Pemulihan job setelah proses mati belum tersedia. |
| 6 | Pengambilan dan parsing diff | Memetakan file serta baris tambahan/penghapusan untuk input analisis dan lokasi komentar. |
| 7 | Indexer struktur Python | Menggunakan `ast.parse` untuk mengekstrak kelas, fungsi, method, serta fungsi test. |
| 8 | Pembentukan relasi kode | Membentuk `CALLS`, `TESTS`, `EXTENDS`, `IMPLEMENTS`, dan `REFERENCES` dengan heuristik nama. Resolusi belum setara analisis tipe atau compiler. |
| 9 | Snapshot graph per commit | Menyimpan versi file, simbol, dan relasi dalam Neo4j berdasarkan repository dan SHA. |
| 10 | Pengambilan konteks dari graph | Mengambil simbol yang berubah dan tetangganya sampai dua lompatan, memberi skor, serta memilih konteks dalam perkiraan anggaran token. |

| No. | Fitur tersedia | Kemampuan dan batas saat ini |
|---|---|---|
| 11 | Integrasi Gemini | Mengirim prompt dan meminta keluaran JSON sesuai schema; memiliki retry serta fallback model pada kondisi HTTP 503 setelah retry. |
| 12 | Instruksi fokus review | Mengarahkan model pada correctness, security, reliability, concurrency, dan maintainability; menghindari masukan yang hanya membahas gaya penulisan. Ini instruksi, bukan jaminan akurasi. |
| 13 | Ringkasan PR dan perubahan per file | Menerbitkan ringkasan, tabel perubahan file, analisis dampak, dan rekap tingkat risiko. Cakupan aktual model belum dilaporkan secara lengkap. |
| 14 | Komentar pada baris perubahan | Memeriksa lokasi temuan sebelum membuat komentar inline. Temuan tanpa lokasi valid dapat masuk ke ringkasan umum. |
| 15 | Suggested changes | Menampilkan usulan penggantian kode dalam format GitHub suggestion bila tersedia. Penerapan dilakukan developer. |
| 16 | Kategori dan tingkat risiko | Menampilkan kategori, severity `low` sampai `critical`, serta confidence dari model. Confidence belum merupakan probabilitas yang terkalibrasi. |
| 17 | Ringkasan yang diperbarui dan deduplikasi temuan | Memperbarui komentar ringkasan bertanda khusus dan memakai fingerprint untuk mengurangi komentar berulang. Belum melacak penyelesaian masalah. |
| 18 | Status proses pada commit | Mengirim `pending`, `success`, atau `error`. `success` menandakan analisis selesai, bukan bahwa kode bebas bug. Menjadikannya syarat merge memerlukan pengaturan repository. |
| 19 | Penyimpanan | Menyimpan temuan, konteks kandidat/terpilih, provider/model, |

| No. | Fitur tersedia | Kemampuan dan batas saat ini |
|---|---|---|
| | catatan review | penggunaan token, serta latensi retrieval dan model. Belum tersedia dashboard atau laporan biaya otomatis. |
| 20 | Konfigurasi global | Environment variable mengatur model, batas file/baris, jumlah temuan, dan parameter graph. Belum ada konfigurasi `.graphmethod.yml` per repository. |
| 21 | Mode dry-run | Melewati publikasi hasil dan status GitHub. Analisis serta pemanggilan Gemini tetap dapat berjalan dan menggunakan token. |
| 22 | Dukungan operasional dasar | Endpoint health, pembacaan secret dari environment/file, Dockerfile, Compose lokal/server, dan Jenkinsfile tersedia. Infrastruktur perusahaan belum diasumsikan memakai konfigurasi yang sama. |
| 23 | Pengujian komponen | Tersedia test untuk diff, indexer, retrieval, review, queue, webhook, keamanan, konfigurasi, serta integrasi API dengan mock. Belum menjadi bukti efektivitas deteksi bug di perusahaan. |

# 5. Fitur yang akan dikembangkan

## 5.1 Dukungan pembentukan graph multi-bahasa

**Kondisi saat ini:** hanya sumber `.py` yang diambil untuk indexing struktural. Diff bahasa lain dapat masuk ke prompt, tetapi belum memperoleh graph bahasa tersebut.

**Pengembangan:** melengkapi indexer Python dengan Tree-sitter untuk bahasa yang disepakati. Kandidat mencakup JavaScript/TypeScript, Go, Java, C++, dan Rust. Pemilihan bahasa pertama mengikuti repository pilot, bukan asumsi mengenai teknologi perusahaan.

Tree-sitter membentuk pohon sintaks konkret dan menyediakan parser untuk beragam bahasa. Penggunaan parser tersebut masih perlu dilanjutkan dengan aturan ekstraksi simbol, pemetaan import, dan resolusi hubungan per bahasa. Karena itu, target yang diajukan adalah

**dukungan multi-bahasa bertahap**, bukan jaminan otomatis memahami semua bahasa atau semua hubungan antarlayanan. Referensi: dokumentasi resmi Tree-sitter.

**Hasil yang ditargetkan:** matriks bahasa yang didukung, jenis simbol/relasi yang dapat diekstrak, batas kemampuan, serta test tiap bahasa. Relasi yang tidak dapat ditentukan tidak dipaksakan menjadi hubungan pasti.

**Kriteria penerimaan:** sampel kode bahasa pilot dapat diindeks; pemanggilan lintas file yang telah ditentukan dalam kasus uji dapat ditemukan; bahasa atau sintaks yang belum didukung dinyatakan dalam laporan cakupan. Dukungan parser tidak dianggap sebagai bukti hubungan semantik sudah benar.

## 5.2 Review ulang melalui komentar `/review`

**Kondisi saat ini:** belum ada pemrosesan perintah komentar, dan deduplikasi job membatasi pekerjaan pada kombinasi repository, PR, dan SHA.

**Pengembangan:** menerima `/review` dari pengguna berwenang pada PR terbuka, mengambil commit terbaru, lalu membuat percobaan review baru tanpa commit tambahan. Perubahan meliputi subscription event komentar, validasi akses, dan penyesuaian identitas percobaan review agar pemintaan ulang pada SHA yang sama dapat dilakukan.

**Manfaat:** developer dapat mengulang review setelah kegagalan sementara atau setelah pengaturan yang sah berubah.

**Kriteria penerimaan:** pengguna yang diizinkan dapat memicu review pada SHA yang sama; event ganda tidak menghasilkan pekerjaan ganda; pengguna tanpa izin tidak dapat memicu pemakaian model; permintaan berulang dibatasi dan mendapat respons status yang jelas.

## 5.3 Konfigurasi per repository melalui `.graphmethod.yml`

**Kondisi saat ini:** konfigurasi tersedia pada tingkat deployment melalui environment variable.

**Pengembangan:** menambahkan konfigurasi repository untuk bahasa komentar, pola file/folder yang diabaikan, fokus review, dan batas temuan. Skema akan divalidasi dan memiliki nilai default. File tidak menyimpan API key atau secret.

**Manfaat:** perilaku reviewer dapat disesuaikan dengan kebutuhan proyek tanpa menjalankan

deployment terpisah untuk setiap repository.

**Kriteria penerimaan:** konfigurasi valid diterapkan secara konsisten; kesalahan konfigurasi dilaporkan; batas resource server tidak dapat dilampaui. Konfigurasi operasional dibaca dari referensi tepercaya seperti base branch, agar perubahan pada PR tidak langsung menonaktifkan pemeriksaan PR itu sendiri.

## 5.4 Laporan cakupan review

**Kondisi saat ini:** laporan menampilkan daftar file berubah, tetapi belum membedakan file yang sepenuhnya masuk ke analisis, terpotong, atau kehilangan konteks struktural.

**Pengembangan:** mencatat dan menampilkan status cakupan per file: dianalisis, dianalisis sebagian, atau dilewati, beserta alasan. Laporan juga membedakan analisis diff dengan ketersediaan graph, menyebutkan SHA, serta menjelaskan batas ukuran atau kegagalan parsing.

**Manfaat:** reviewer memahami bagian yang tetap perlu diperiksa manual. Istilah “dianalisis” menjelaskan input yang diproses, bukan jaminan semua bug ditemukan.

**Kriteria penerimaan:** setiap file pada diff memiliki status; pemotongan prompt tidak disamarkan sebagai review penuh; bahasa tanpa graph diberi keterangan yang tepat; cakupan dihitung oleh pipeline, bukan hanya berdasarkan klaim model.

## 5.5 Pelacakan penyelesaian temuan antarcommit

**Kondisi saat ini:** fingerprint mengurangi duplikasi komentar, tetapi belum memiliki identitas temuan yang stabil ketika baris bergeser atau judul keluaran AI berubah.

**Pengembangan:** menghubungkan temuan dengan simbol, lokasi, kategori, dan bukti masalah antarversi. Status yang diusulkan: baru, masih terdeteksi, terindikasi diperbaiki, dan perlu verifikasi. Ringkasan memperlihatkan perubahan status dan commit terkait.

**Manfaat:** developer dapat mengikuti tindak lanjut masalah tanpa menelusuri semua komentar lama secara manual.

**Kriteria penerimaan:** pergeseran baris tidak otomatis menghasilkan temuan baru; masalah yang tetap ada dipertahankan; temuan yang tidak muncul karena file dilewati, konteks

terpotong, atau analisis gagal tidak ditandai selesai. Hilangnya keluaran AI saja tidak cukup sebagai bukti perbaikan.

## 5.6 Analisis kode sebelum dan sesudah perubahan

**Kondisi saat ini:** retrieval menggunakan snapshot head commit. Graph versi sebelum perubahan belum digunakan dalam alur review.

**Pengembangan:** mengambil konteks dari versi dasar dan versi terbaru yang konsisten dengan diff. Untuk PR, versi sebelum perubahan perlu mengikuti titik pembanding diff, termasuk merge base bila relevan, bukan sekadar mengambil tip branch dasar saat itu.

Sistem akan memeriksa perubahan signature, penghapusan atau pemindahan simbol, relasi pemanggil, serta pewarisan yang terdampak. Prompt membedakan sumber konteks lama dan baru agar model tidak mencampurkan keduanya.

**Manfaat:** menyediakan konteks untuk menilai potensi breaking change, khususnya ketika simbol yang dihapus sudah tidak ditemukan pada graph terbaru.

**Kriteria penerimaan:** kasus penghapusan fungsi, perubahan parameter, dan perubahan inheritance dapat mengambil konteks yang diharapkan dari kedua versi; setiap konteks memiliki identitas commit; perubahan PR selama analisis tidak menghasilkan publikasi yang menyatakan hasil lama sebagai hasil terbaru.

# 6. Penguatan keandalan sebelum pilot

Enam fitur di atas perlu disertai perbaikan pada alur yang sudah ada agar pilot dapat dipercaya.

| Pekerjaan | Hasil yang diperlukan |
|---|---|
| Pemulihan job setelah worker mati | Pekerjaan yang tertinggal dalam status `processing` dapat dipulihkan dengan batas waktu kepemilikan job dan pengamanan agar tidak dipublikasikan ganda. |
| Konsistensi commit | Metadata, diff, dan snapshot merujuk versi yang sesuai; pemeriksaan ulang dilakukan sebelum publikasi. |

| Pekerjaan | Hasil yang diperlukan |
|---|---|
| Validasi bukti temuan | Path dan referensi konteks diperiksa; temuan tanpa dukungan memadai diberi perlakuan yang jelas. |
| Publikasi dan pagination | Komentar lama tetap ditemukan pada PR dengan banyak komentar; retry publikasi tidak menimbulkan duplikasi yang tidak perlu. |
| Retensi dan operasional | Masa simpan snapshot/catatan ditetapkan, prosedur backup-restore diuji, serta job gagal dan antrean macet dapat diketahui pengelola. |

# 7. Batas ruang lingkup

Pengembangan berfokus pada GitHub PR dan bahasa repository pilot. Sistem memberi analisis berbasis diff serta konteks yang berhasil diperoleh; sistem tidak menjamin menemukan seluruh bug, memahami seluruh perilaku runtime, atau menggantikan pengujian, pemeriksaan tipe, dan reviewer manusia.

Pemindaian dependency vulnerability, eksekusi test milik PR, perbaikan kode otomatis, auto-merge, dashboard khusus, dan integrasi GitLab/Bitbucket belum termasuk dalam enam fitur yang diajukan. Penambahan tersebut dapat dibahas terpisah apabila pilot menunjukkan kebutuhan nyata.

Default codebase saat ini membatasi PR menjadi 30 file berubah dan 2.000 baris tambah/hapus, serta maksimal 500 file Python yang memenuhi syarat indexing. Nilai dapat diubah, tetapi menaikkan batas tidak otomatis membuat indexing efisien. Review PR besar secara bertahap belum termasuk implementasi saat ini maupun komitmen inti proposal.

# 8. Pengelolaan akses dan data perusahaan

Pilot hanya memakai repository yang ditetapkan SMP Technology dan GitHub App dengan izin sesuai kebutuhan. Perintah manual harus memeriksa hak pengguna. Perubahan izin/subscription untuk fitur komentar akan didokumentasikan saat implementasi.

Pemanggilan Gemini mengirimkan diff, metadata PR, dan konteks kode terpilih ke layanan model. Hosting API, worker, PostgreSQL, dan Neo4j pada server perusahaan tidak membuat

seluruh pemrosesan menjadi lokal. Perusahaan perlu menetapkan repository serta kategori data yang boleh diproses dan memastikan penggunaan provider sesuai kebijakan internal sebelum pilot memakai kode tersebut.

Secret disimpan di mekanisme pengelolaan secret deployment, tidak di konfigurasi repository. Log tidak perlu memuat source code atau prompt mentah. Snapshot Neo4j menyimpan isi simbol kode; catatan review dan webhook juga dapat memuat informasi internal, sehingga akses, backup, dan retensinya perlu dikelola.

Kode PR diperlakukan sebagai data untuk dianalisis. Parser tidak perlu mengimpor atau menjalankan kode tersebut. Instruksi review saat ini meminta model mengabaikan instruksi di dalam source, tetapi hal itu belum merupakan jaminan perlindungan prompt injection; skenario tersebut perlu masuk pengujian pilot.

# 9. Tahapan implementasi dan hasil kerja

| Tahap | Pekerjaan | Hasil yang diserahkan |
|---|---|---|
| 1. Penetapan pilot | Memilih repository, bahasa awal, PIC, akses data, dan batas penggunaan. Memeriksa apakah GitHub sesuai alur tim. | Ruang lingkup dan kriteria penerimaan yang disepakati. |
| 2. Verifikasi dasar | Menyiapkan lingkungan uji dan menjalankan test/integrasi; memperbaiki job macet, konsistensi SHA, dan publikasi. | Baseline sistem beserta hasil uji keandalan. |
| 3. Kendali penggunaan | Mengimplementasikan `/review`, konfigurasi repository, dan laporan cakupan. | Alur review yang dapat dikendalikan dan diperiksa developer. |
| 4. Perluasan konteks | Menambahkan bahasa pilot menggunakan Tree-sitter dan analisis versi sebelum/ sesudah. | Matriks dukungan bahasa, graph, dan kasus uji dampak lintas file. |
| 5. Tindak lanjut temuan | Menghubungkan temuan antarcommit dan menyajikan perkembangannya. | Status temuan dan bukti pengujian skenario perubahan kode. |

| Tahap | Pekerjaan | Hasil yang diserahkan |
|---|---|---|
| 6. Pilot dan evaluasi | Mengoperasikan sistem pada PR terpilih, menilai temuan bersama reviewer, dan mencatat kendala. | Laporan hasil pilot, dokumentasi penggunaan, serta rekomendasi kelanjutan. |

Durasi dan tanggal tiap tahap ditentukan setelah jumlah bahasa, kondisi repository, kapasitas pengembang, serta akses lingkungan diketahui. Tabel ini merupakan urutan kerja yang diusulkan, bukan komitmen jadwal yang sudah disetujui.

# 10. Pengukuran keberhasilan

Keberhasilan diukur dari hasil yang dapat diperiksa, bukan banyaknya komentar yang dihasilkan AI. Nilai target ditetapkan bersama tim sebelum pilot agar evaluasi tidak berubah mengikuti hasil.

| Aspek | Cara evaluasi |
|---|---|
| Ketepatan temuan | Reviewer manusia menilai temuan valid dan keliru. Precision dihitung dari jumlah temuan valid dibagi seluruh temuan yang dinilai. |
| Bug yang berhasil ditemukan | Gunakan kasus uji dengan bug yang sudah diketahui untuk mengukur recall. PR biasa tanpa daftar bug acuan tidak cukup untuk menghitung recall. |
| Manfaat konteks graph | Sebagai eksperimen evaluasi, bandingkan diff-only dengan diff + graph pada PR dan model yang sama. Mode diff-only untuk eksperimen masih perlu ditambahkan; konfigurasi saat ini hanya menerima graph. |
| Waktu | Catat waktu tunggu antrean, durasi total sampai review terbit, serta latensi model/retrieval. Dua metrik terakhir sudah memiliki pencatatan dasar. |
| Keandalan | Uji restart worker, webhook ganda, kegagalan provider, dan commit baru saat review berlangsung. |
| Transparansi | Pastikan file yang dilewati atau input yang terpotong tercantum dalam |

| Aspek | Cara evaluasi |
|---|---|
| cakupan | laporan. |
| Penggunaan resource | Catat token, request, dan pertumbuhan penyimpanan; hitung biaya memakai tarif provider yang berlaku saat pilot. |
| Kegunaan bagi developer | Kumpulkan penilaian singkat mengenai relevansi temuan, kemudahan tindak lanjut, dan beban komentar. |

Evaluasi awal berjalan sebagai bantuan review tanpa menjadikan hasil AI sebagai penentu otomatis merge. Keputusan memperluas penggunaan diambil setelah melihat hasil, keterbatasan, serta beban operasional pilot.

# 11. Kebutuhan sumber daya dan pembagian peran

| Kebutuhan | Usulan |
|---|---|
| Repository | Repository pilot dan contoh PR yang diizinkan untuk pengembangan/ evaluasi. |
| Lingkungan | Server atau lingkungan container untuk API, worker, PostgreSQL, dan Neo4j, dengan endpoint HTTPS webhook. Ukuran resource ditentukan dari beban pilot. |
| Akses integrasi | GitHub App, akses deployment sesuai tugas, serta kredensial layanan model melalui mekanisme secret. |
| Pengembang magang | Implementasi fitur, pengujian, dokumentasi, pencatatan hasil, dan perbaikan selama pilot. |
| Pembimbing/ engineering lead | Menetapkan prioritas, meninjau perubahan, mengonfirmasi batas akses, dan menilai hasil pilot. |
| Reviewer pilot | Memverifikasi temuan AI dan memberi masukan pada penggunaan sehari-hari. |

| Kebutuhan | Usulan |
|---|---|
| PIC operasional | Membantu deployment, pemantauan, backup, dan prosedur penghentian pilot bila diperlukan. |

Komponen biaya meliputi komputasi aplikasi/database, penyimpanan dan backup, penggunaan API model, serta waktu implementasi dan evaluasi. Nominal belum ditetapkan karena volume PR, ukuran konteks, model yang disetujui, dan fasilitas perusahaan belum diketahui. Tidak diperlukan asumsi pembelian layanan baru sebelum fasilitas yang tersedia diperiksa.

# 12. Usulan keputusan kepada SMP Technology

Saya mengajukan persetujuan untuk menjalankan pengembangan dan pilot terbatas GraphMethod dengan enam fitur yang dijelaskan dalam proposal ini. Dukungan yang diperlukan adalah penetapan pembimbing/PIC, repository dan bahasa awal, lingkungan pengujian, ketentuan pengiriman kode ke provider model, serta batas penggunaan resource.

Hasil akhir yang diserahkan mencakup implementasi fitur yang disepakati, dokumentasi instalasi dan penggunaan, matriks kemampuan bahasa, hasil pengujian, serta laporan evaluasi pilot. Penggunaan lebih luas ditentukan bersama berdasarkan bukti manfaat dan kesiapan operasional.

# Lampiran: Dasar verifikasi status codebase

| Area | Berkas rujukan |
|---|---|
| Webhook dan keamanan | webhooks.py, security.py |
| Antrean dan worker | queue.py, worker.py |
| Pengambilan kode dan publikasi GitHub | github.py |
| Parsing dan graph | indexer.py, graph.py, retrieval.py |
| Prompt, validasi, dan hasil review | review.py, base.py |

| Area | Berkas rujukan |
| --- | --- |
| Provider model | gemini.py, registry.py |
| Konfigurasi dan penyimpanan | config.py, models.py |
| Pengujian dan deployment | tests, Dockerfile, Jenkinsfile, Compose lokal, Compose server |

Dokumen disusun dari pembacaan codebase dan dokumentasi, bukan hasil deployment atau benchmark baru. Belum ada klaim penghematan waktu, tingkat akurasi, atau kesiapan produksi yang dibuktikan pada lingkungan SMP Technology. Nama model pada konfigurasi perlu diverifikasi ketersediaannya saat penyiapan pilot; proposal tidak menjanjikan model tertentu.